1 Introduction

In this lab, we will be working with the CORDIC algorithm and learn how to implement, simulate and synthesize a CORDIC module using a High-Level Synthesis (HLS) language (C/C++). These tasks will be carried out using the **Vitis HLS** tool.

1.1 Introduction to CORDIC

CORDIC is an acronym for Coordinate Rotation Digital Computer introduced by Volder in 1959. Originally intended for the computation of trigonometric functions, this simple and efficient algorithm was later generalized to include hyperbolic functions, multiplication, and division. This iterative technique computes multiplicative and similar functions by shift and add operations. It is commonly used in FPGAs where no hardware multiplier is present.

If we were to have a brief mathematical overview of the method:
Updating 3 variables, say *x*, *y*, and *z,* in the following manner,

$$x_{k+1} = x_k - d_k 2^{-k} y_k$$

$$y_{k+1} = y_k + d_k 2^{-k} x_k$$

$$z_{k+1} = z_k - d_k tan^{-1}\left(2^{-k}\right),$$

where $d_k = sign(z_k) = \{-1, \ if \ z_k < 0 \ \ +1, \ if \ z_k \geq 0$.

After a sufficient number of iterations, *x*, *y* and *z* converge to:

$$x_k = G(x_0 cosz_0 - y_0 sinz_0)$$

$$y_k = G(x_0 sinz_0 - y_0 cosz_0)$$

$$z_k = 0,$$

where *G=1.64676.*

If we choose x_0 and y_0 properly, we can infer the value of $cos(z_0)$ and $sin(z_0)$ from x_k and y_k.
For example, if we choose $x_0 = \frac{1}{G}$, $y_0 = 0$, we have $x_k = \cos cos\left(z_0\right)$, $y_k = \sin sin\left(z_0\right)$.

Similarly, when we set d_k as $d_k = - sign(y_k)$
After a sufficient number of iterations, *x*, *y* and *z* converge to:

$$x_k = G\sqrt{x_0^2 + y_0^2}$$

$$y_k = 0$$

$$z_k = z_0 + \tan^{-1}\left(\frac{y_0}{x_0}\right)$$

If we choose x_0, y_0 and z_0 properly, we can infer the tan^{-1} function.

If we implement this algorithm in hardware, the schematic of the design would be the one shown in Figure 1.

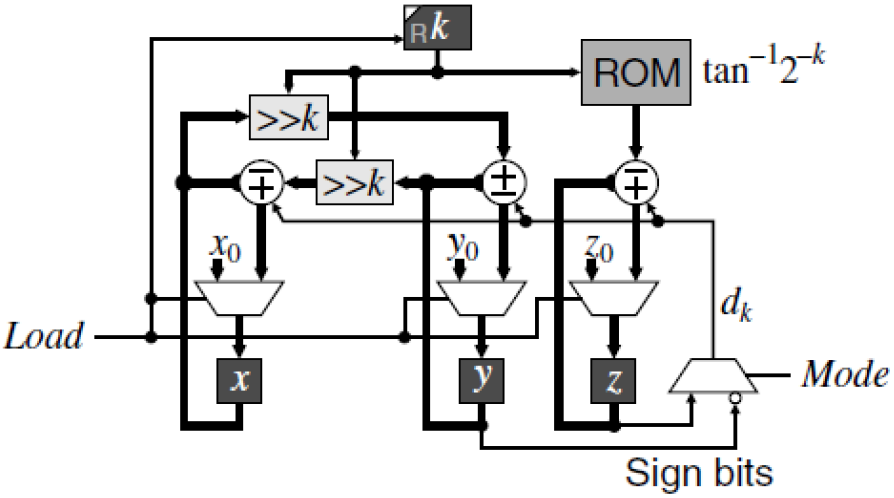


Figure 1: CORDIC Module Schematic.

2 HLS code for CORDIC

This section provides a brief overview of the steps needed for implementing a CORDIC module. Detailed step by step guide for Vitis HLS can be found in section 3.

Please download `Lab1_code.zip` from Teams (Lab1 Channel Files) and extract them before you proceed. In the extracted folder, you will find three files named as `basic_cordic.h`, `basic_cordic.cpp`, and `basic_cordic_tb.cpp`. These files contain the CORDIC implementation of the sin and cos function that we will be using to learn the process of HLS verification, simulation and synthesis. Please refer to section 3 for detailed steps.

Please make sure the functionalities of your design (basic_cordic.cpp) meet the following requirements:

1) The *cordic_sin_cos* module calculates *sin* and *cos* with initial input x_0, y_0, and z_0.
2) The *cordic_arctan* module calculates tan^{-1} with input sine and cosine values. You need to specify x_0, y_0 and z_0 with given inputs.
3) For the calculation, please use a 16-bit signed fixed-point number with 12 bits for the fraction.

4) Please make sure the errors of your results are less than 0.05.

Please complete the blank part in `basic_cordic.cpp` and use the provided testbench (`basic_cordic_tb.cpp`) to check the correctness. In the testbench, *test_sin_cos* tests your implementation of the *cordic_sin_cos* module, and *test_arctan* checks the correctness of the *cordic_arctan* module.

1) In *test_sin_cos*, x_0, y_0 and z_0 are set as 1, 0 and θ, where θ is the random input angle in the range $[-\pi, \pi]$.
2) In *test_arctan*, 10 angles are used to check the correctness of your code. If your implementation is correct, the output of the *cordic_arctan* module should be almost the same as the provided angles.

Once you pass the testbench, we will do the behavior simulation to verify our design.

3 Implementation of CORDIC circuit using Vitis HLS

In this lab, we will implement the hardware design of a CORDIC module using **Vitis HLS** and synthesize the C/C++ code of a CORDIC function to Verilog without directly implementing it in Verilog. Please go through the following steps.

3.1 Preparing files and tools on Olympus

a) These steps only apply to those using Olympus server. If you are using Vitis on your personal computer, please move on to section 3.2.

b) Open a terminal (MacOS) or mobaXterm (windows)

c) The Lab1_code.zip file needs to be transferred over from your personal computer to the Olympus server. In the terminal run the following command to transfer:

```
cd </path/to/Lab1_code.zip>
scp Lab1_code.zip <USER_NAME>@olympus.ece.tamu.edu:~
```

Replace <USER_NAME> with your Olympus user name. Don't forget the ~ at the end. The zip file will be downloaded in your Olympus home directory.

d) Now login into Olympus using steps from Lab0. Once logged in, open MobaXterm and unzip the Lab1_code.zip file.

```
unzip Lab1_code.zip
```

e) Load a ECEN module and Enable Vivado/Vitis HLS by running the following command:

```
load-ecen-248
source /opt/coe/Xilinx/Vivado/2023.1/settings64.sh
```

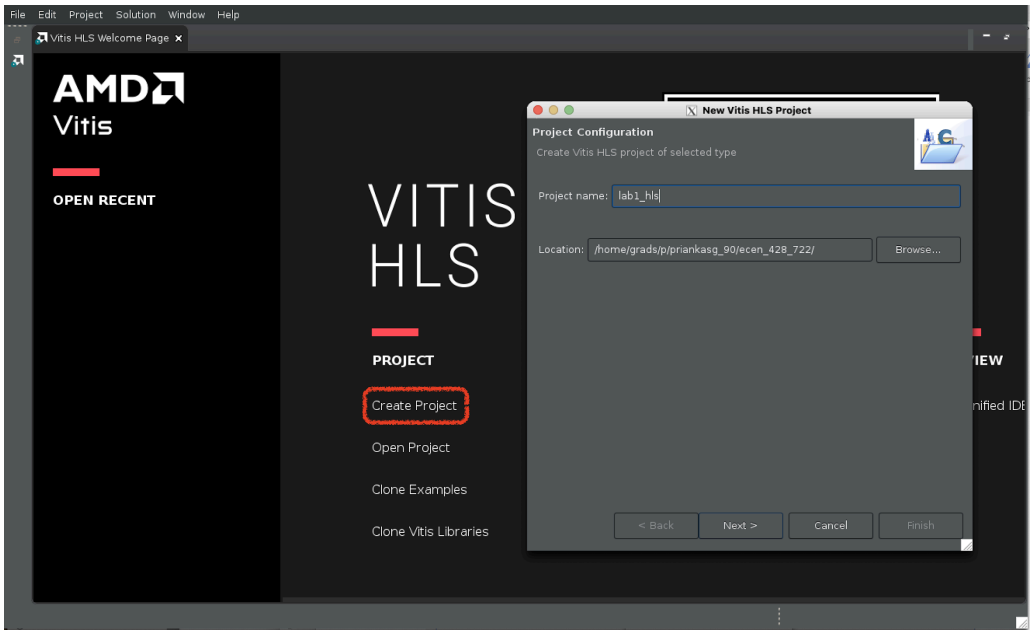
f) Launch Vitis HLS by running the following command:

```
vitis_hls
```

g) To speed up creating and opening projects, change this setting:
Window □ Preferences □ Expand C/C++ □ Indexer □ uncheck Enable Indexer

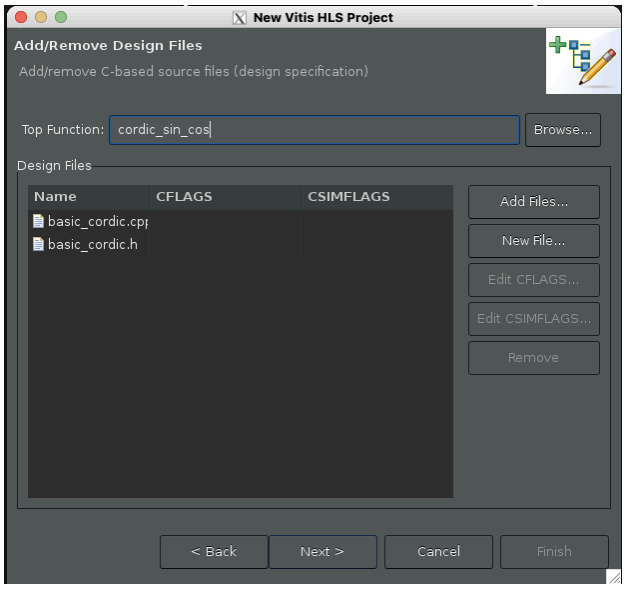
3.2 Design Flow in Vitis HLS

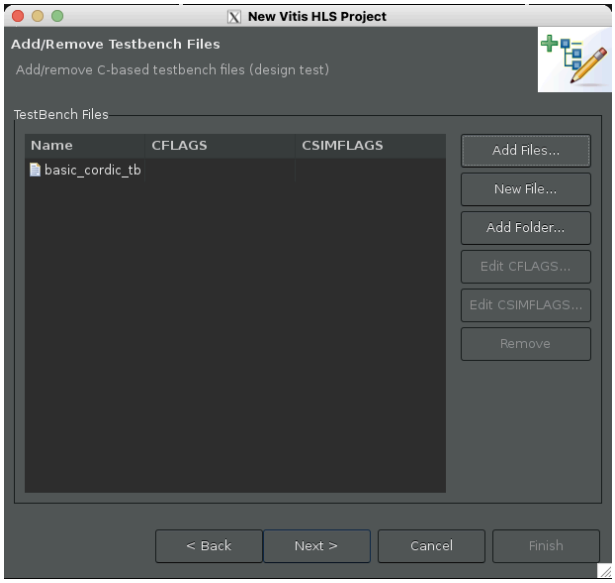
a) Once Vitis HLS is loaded, click on "Create New Project" to create a new project. Enter the project name with your preference on the welcome page. Then, click on "**Next**".



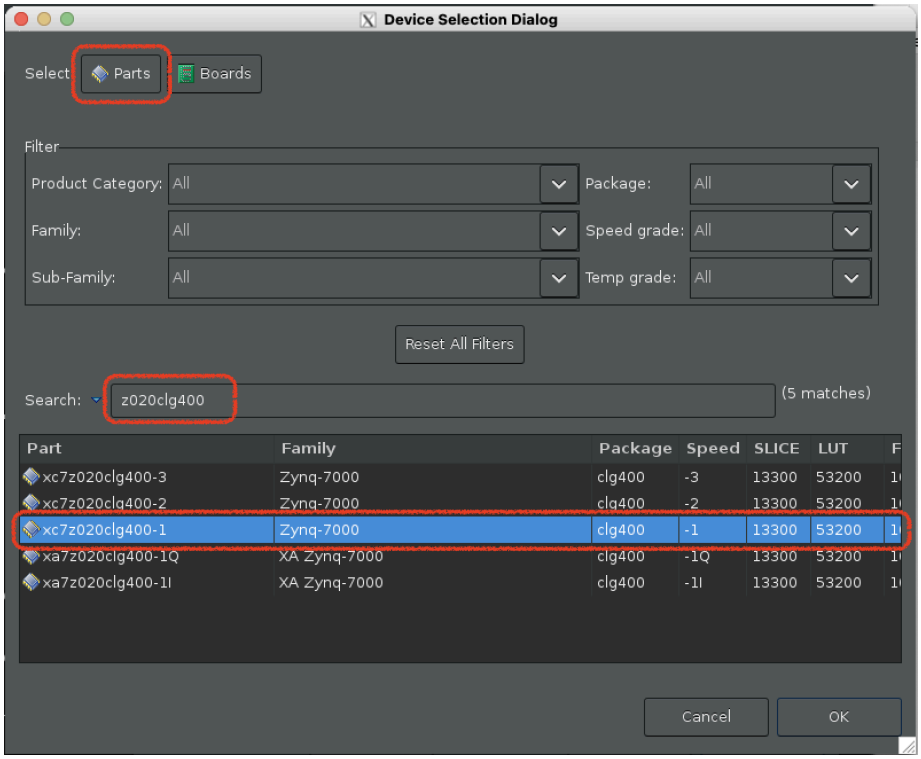
b) In the next window, add `basic_cordic.cpp` and `basic_cordic.h` in the project by clicking on "**Add Files…**". Then type *cordic_sin_cos* as the top function, (Note that we will switch to cordic_arctan later for testing implementation of *arctan*.) click on "**Next**".

c) Add "basic_cordic_tb.cpp" in the Testbench Files and click on "**Next**".

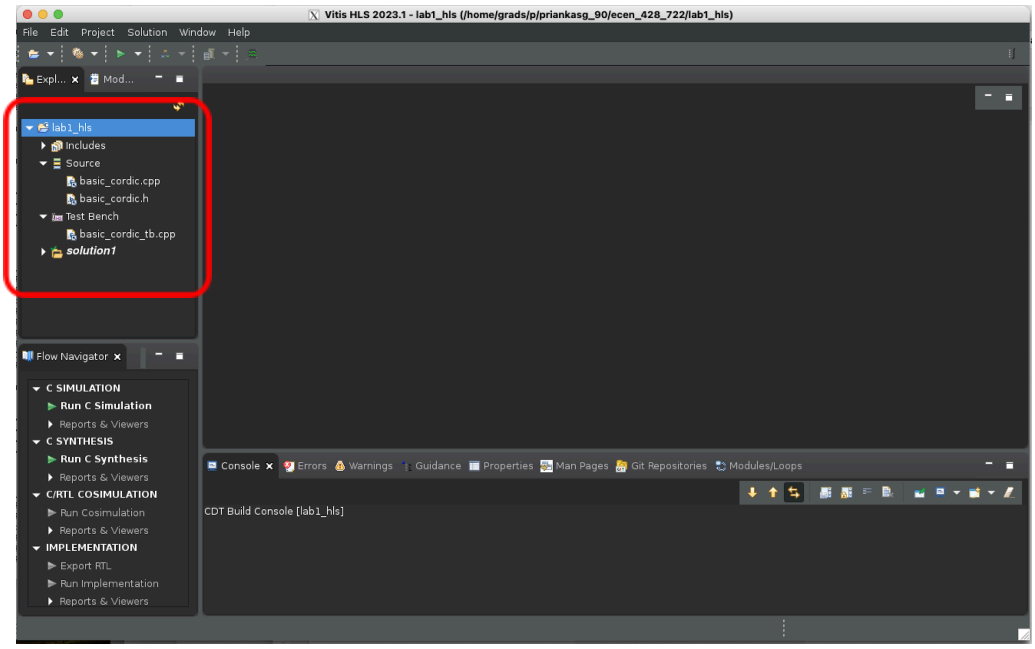




d) Click on "**…**" in the Part Selection block, choose "**Parts**", select "**xc7z020clg400-1**" and click on "**OK**. Choose "**OK**" and "**Finish**" after you are done.



e) After the project is successfully created, you should be able to see the source code and testbench on the left column.



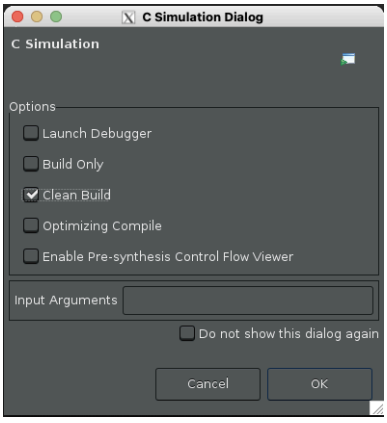
Now the project should be properly configured for you to work on. We will continue to work on this project in the next section.

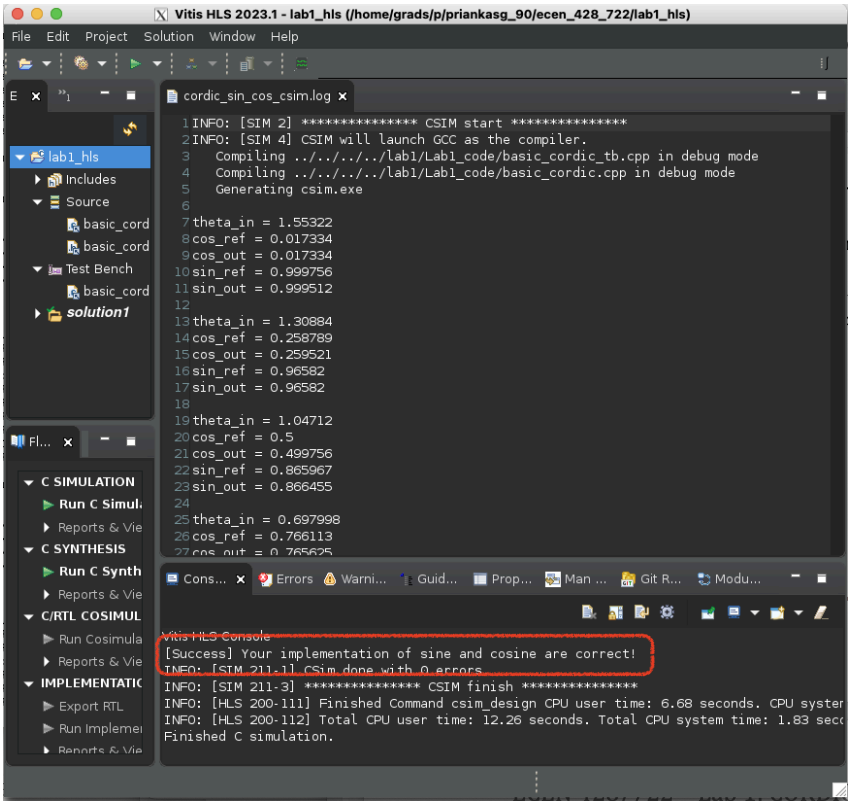
3.3 Run C++ Verification

In this section, we will show how to verify the correctness of your implementation in Vitis HLS by running C Simulation.

a) On the main page, click on "**Project** -> **Run C Simulation**". Check the box of "**Clean Build**", and then click "**OK**".

b) Wait until the simulation is done. If your implementations are all correct, you should see "*[Success] Your implementation of xxx is correct!*" in the console.



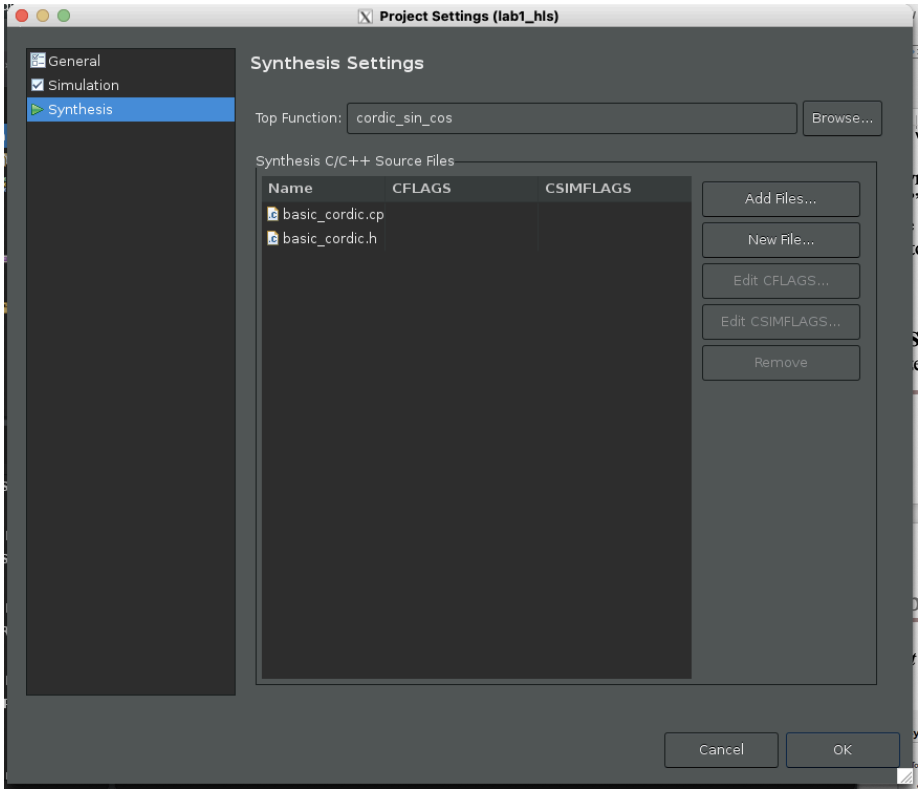


Now we are ready to synthesize our C++ implementation to RTL. Please go through the following steps in Section 3.3.

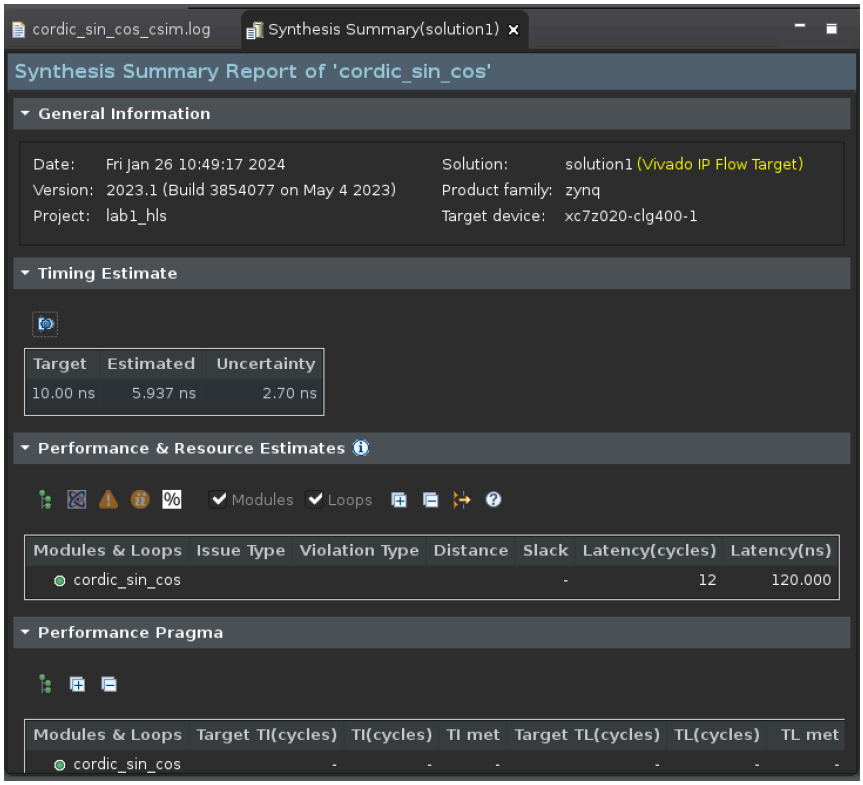
3.4 Steps for C/RTL Co-simulation

Once you pass the C++ verification, we can start synthesizing the code to hardware language (RTL).

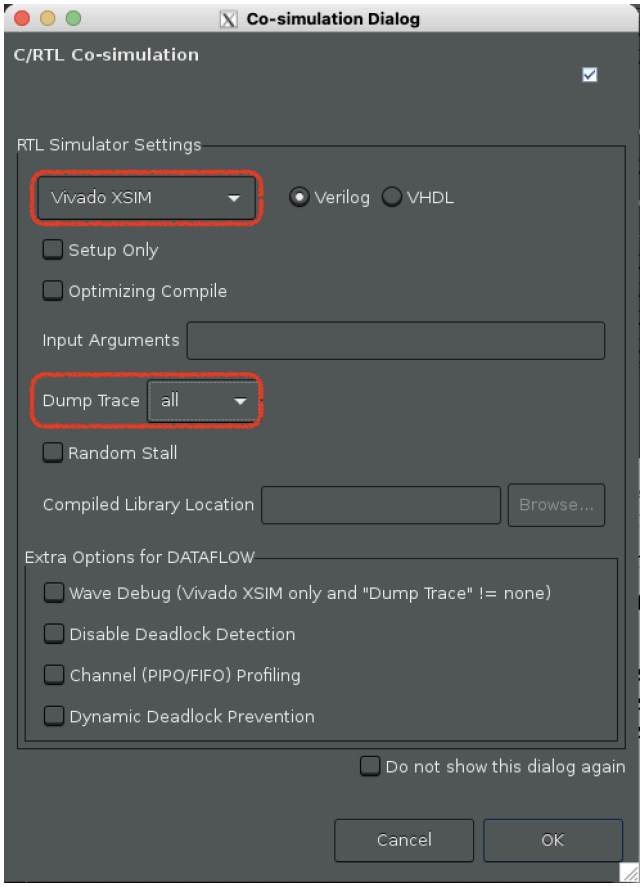
a) Before we start the C++ Synthesis, the top function should be specified for HLS. Click on "**Project -> Project Settings**", choose "**Synthesis**" on the left column. Browse the target function you want to synthesize to the RTL module. This should be cordic_sin_cos at the moment. Later you will need to change this to *cordic_arctan* for the submission task of this lab.



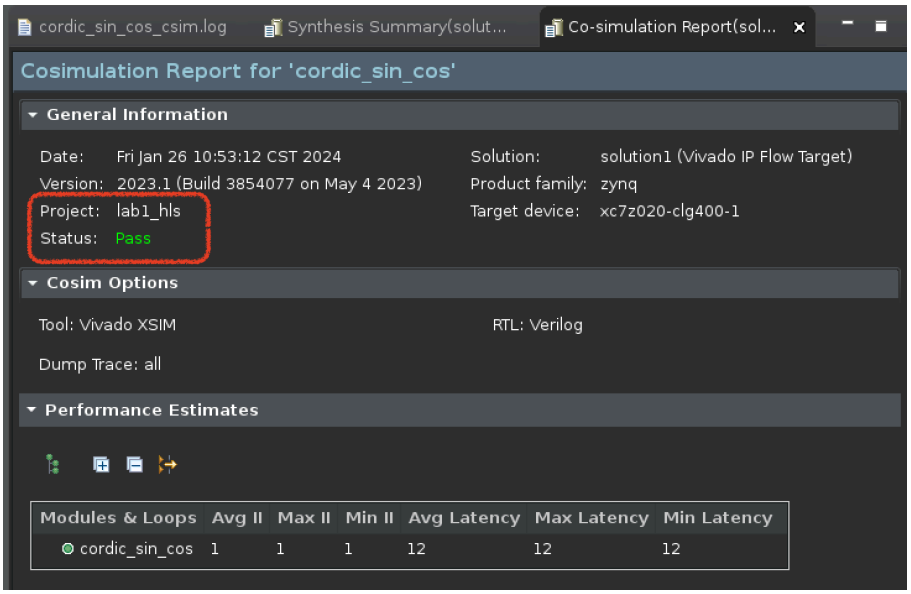
b) Select "**Solution -> Run C Synthesis -> Active Solution**". Click **"OK".** Vitis will start to synthesize your code to RTL implementation. If everything works well, you should see a *Synthesis Report* show up.



c) Now we are ready to perform the behavior simulation of the generated RTL implementation. Please select "**Solution -> Run C/RTL Co-simulation**". Set the simulator to **Vivado XSIM**, and make sure to set "**all**" for Dump Trace.



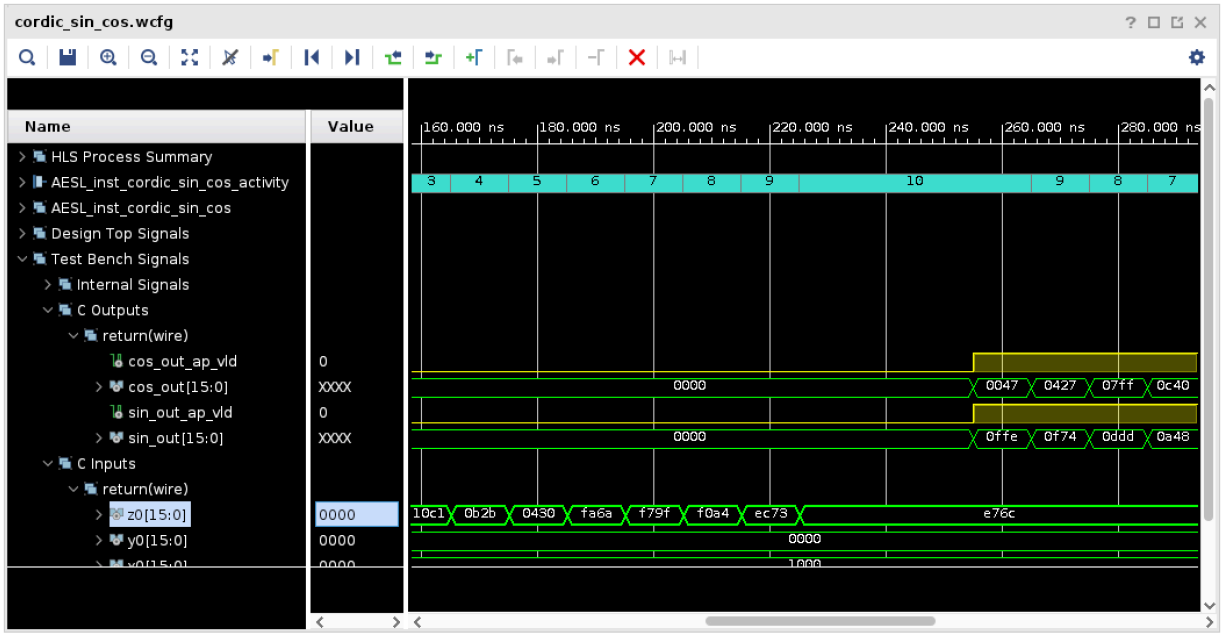
d) Once the co-simulation is done, you should find a *Co-simulation Report* that show up in the window. Make sure the status of the Verilog result is *Pass*!



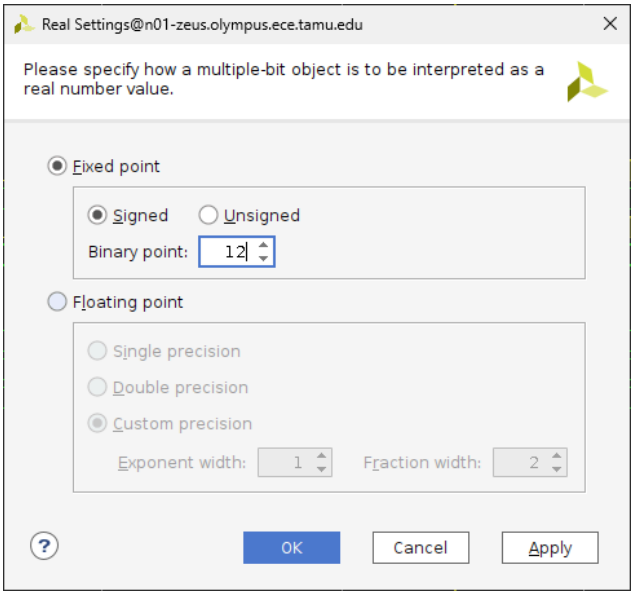
e) Now the behavior simulation of your module is ready. Please find "**Open Wave Viewer…**" on the top bar ( ) and click it. If the icon does not appear on the bar, go back to step c and make

sure the configuration is correct.

f) It may take a little time, but Vivado will launch and show the simulation waveforms.



g) Note that we can convert the hexadecimal results to decimal values. Right click on the value and choose "**Radix -> Real Settings**".



4 Implementation of hyperbolic CORDIC design

a) Please create a new project. Add "hyperbolic_cordic.cpp", "hyperbolic_cordic.h", and "hyperbolic_cordic_tb.cpp" to your project.

b) Please implement the cordic_sinh_cosh to calculate sinh and cosh in "hyperbolic_cordic.cpp"

c) Please run the behavioral simulation using the C/RTL Co-simulation tool.

5 Pre-lab submission

1. Please only submit 1 PDF file, containing the following items:

    a) Screenshots of your behavior simulation of CORDIC-based sine and cosine (similar to Figure 10).

    b) A few words explaining the results in the screenshots.

2. Please name the PDF file as "Lab#_PreLab_Section#_LastName_FirstName.pdf".

3. Please submit the PDF file on Canvas.

6 Post-lab submission

1. Please submit a zip file, containing all the code files you used in this lab ("Lab#_PostLab_Section#_LastName_FirstName.zip")

2. Submit a PDF report and name it as "Lab#_PostLab_Section#_LastName_FirstName.pdf".

3. In the PDF report, please include the following items:

    a. Screenshots of the behavioral simulations for CORDIC-based tan^{-1}.

    b. Screenshots of the behavioral simulations for hyperbolic CORDIC.

    c. A few words explaining the results in the screenshots.

4. Please submit both zip and PDF files on Canvas.